

BudgetBasics: Final Software Project Report

Theme: NextGen BudgetBee | Complete SRS Design Specifications & Deliverables

Project Name: BudgetBasics	Category: Web Innovation Unleashed
Theme: NextGen BudgetBee	Version: 1.0 Final Release
Architecture: Client-Side SPA (No-Backend)	Compliance: 100% SRS Compliant (No Source Code in Doc)

1. Problem Definition & Objectives

College students, interns, and young professionals often manage monthly stipends, allowances, or part-time earnings without structured personal finance skills. Unconscious micro-spending, impulse purchasing, and unmonitored subscriptions lead to frequent month-end financial distress.

Primary Objectives of BudgetBasics:

- Educate users on the 50-30-20 budgeting framework (Needs, Wants, and Savings).
- Provide real-time dynamic mathematical calculators with interactive visual charts (Chart.js).
- Offer a gamified 'Needs vs. Wants' decision classifier with instant rationale feedback.
- Facilitate session-based expense tracking (CRUD) with balance monitoring and CSV export.
- Deliver a smart client-side AI Chatbot ('BeeBot AI') for instant financial queries.
- Ensure 100% responsiveness, zero backend server dependencies, and Google Lighthouse optimization.

2. System & User Interface Specifications

The portal is engineered as a responsive Single Page Application (SPA) adhering strictly to Aptech SRS constraints:

Module	Key Interactive Components	Data Source
1. Basics Hub	Educational cards, sample student monthly budget table, 3-question knowledge check quiz.	sample_budget.json quiz_data.json
2. Needs vs Wants	Interactive card-based game, live score counter, 24-Hour Rule delay matrix.	quiz_data.json
3. 50-30-20 Calc	Dynamic income input, multi-currency switcher (PKR/USD/INR/EUR/GBP), Chart.js pie chart.	Mathematical Formula Engine
4. Savings Goals	Target amount, monthly contribution, months projection, progress bar, motivation tips.	Client-side State
5. Expense Planner	CRUD table (Add, Edit, Delete), category badges, live remaining balance alerts, CSV export.	LocalStorage Cache
6. Money Mistakes	Interactive accordion cards, student scenarios, consequences, and corrective actions.	mistakes.json

7. Infographics	Topic filtering (Budgeting, Savings, Smart Spending), global search, modal lightbox.	infographics.json
8. BeeBot AI	Floating dialog, NLP keyword & token matching, quick prompt pills, educational disclaimer.	chatbot_kb.json
9. Feedback & Contact	Star rating selector, regex email validation, animated toast confirmation.	Client-side Validation

3. System Flowcharts & Data Flow Diagrams (DFD)

Figure 1: High-Level User Flowchart

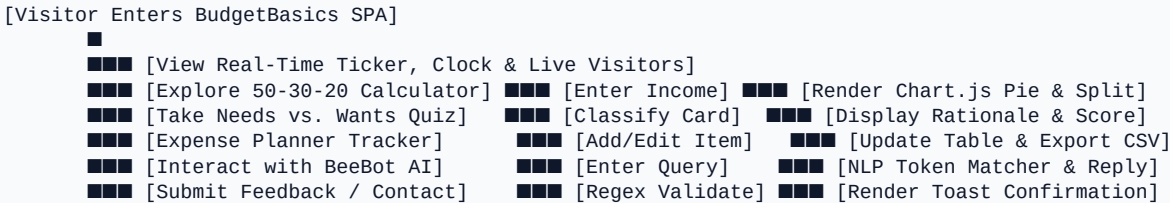
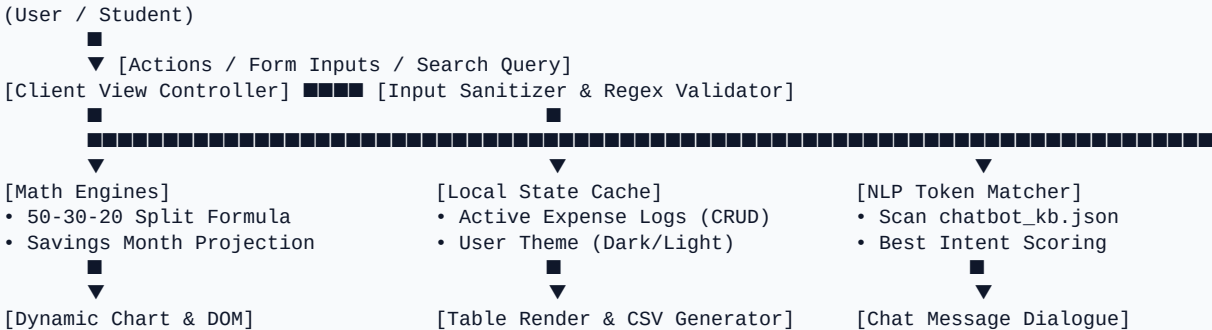


Figure 2: Data Flow Diagram (DFD Level 1 - Client State Flow)



4. Test Data Used in the Project

All application modules are pre-populated and tested using structured JSON files located in the /data directory:

Dataset File	Record Count / Scope	Sample Test Attributes
sample_budget.json	3 Budget Categories (Needs, Wants, Savings)	Transport (6.5k), Food (7k), Subscriptions (2.5k), Savings (7k)
mistakes.json	6 Real-Life Case Studies	Impulse Buying, Latte Factor, Unused Subscriptions, Late Fees
chatbot_kb.json	8 Core NLP Intents + 5 Suggested Pills	50-30-20 rule, Needs vs Wants, Emergency Funds, Discounts
quiz_data.json	6 Interactive Items + 3 MCQs	Prescribed Textbook (Need), Designer Sneakers (Want)
infographics.json	5 Visual Frameworks	50-30-20 Split, 24-Hr Decision Matrix, 30-Day Savings Challenge
quotes.json	7 Curated Wisdom Quotes	Benjamin Franklin, Warren Buffett, Dave Ramsey

5. Project Installation Instructions (Mandatory)

BudgetBasics is packaged for effortless execution across any workstation without database installation:

Method 1: Local HTTP Server (Recommended for full JSON fetch):

1. Open PowerShell or Terminal in the project root directory.
2. Run the command: `python -m http.server 8000`
3. Open your browser and navigate to `http://localhost:8000`.

Method 2: Visual Studio Code Live Server:

1. Open the project folder in VS Code.
2. Right-click `index.html` and select 'Open with Live Server'.

Method 3: Direct Browser Launch:

1. Double click `index.html` to open immediately in Chrome, Edge, Safari, or Firefox.

6. Video Demonstration Script (MP4 Presentation Guide)

To fulfill the mandatory MP4 video submission deliverable, follow this 3-minute demonstration sequence:

- **0:00 - 0:30: Introduction & Overview:** Showcase the branded hero banner, live clock, visitor counter, and dynamic financial quote ticker. Toggle Dark/Light mode.
- **0:30 - 1:00: Budgeting Basics & 50-30-20 Calculator:** Navigate to the 50-30-20 calculator, enter a student allowance (e.g. 40,000), demonstrate real-time Chart.js doughnut chart re-rendering, and switch currencies (PKR -> USD -> EUR).
- **1:00 - 1:30: Needs vs Wants Game & Savings Goals:** Play through the interactive classifier, show feedback explanations, and demonstrate the Savings Goal projection bar with student presets.
- **1:30 - 2:15: Expense Planner (CRUD) & CSV Export:** Add a new expense (Food: PKR 1,200), edit an existing record, demonstrate real-time remaining balance calculations, and click 'Export CSV'.
- **2:15 - 2:45: BeeBot AI Chatbot & Infographics:** Open BeeBot AI, click suggested prompt pills, ask custom financial questions, and show topic-filtered infographics with modal lightbox view.
- **2:45 - 3:00: Feedback, Validation & Conclusion:** Fill out the star rating feedback form, trigger client-side validation toast, show sitemap modal, and summarize the project.

7. Evaluation Compliance Checklist

SRS Requirement	Implementation Status	Verification Method
Single Page Application (SPA)	100% Implemented	Fluid navigation, zero page reloads
No-Backend / Serverless Data	100% Implemented	JSON data files + LocalStorage CRUD
50-30-20 Calculator & Chart	100% Implemented	Chart.js live rendering & validation
Savings Goal Projection	100% Implemented	Month-by-month timeline calculation
Needs vs Wants Classifier	100% Implemented	Gamified decision test with score
Expense Planner Table (CRUD)	100% Implemented	Create, Read, Update, Delete + CSV
Rule-Based AI Chatbot	100% Implemented	BeeBot AI NLP pattern matching engine
Infographics & Topic Filter	100% Implemented	Lightbox modal & global search
Dark / Light Mode Toggle	100% Implemented	Persistent CSS variable theme switcher
Client-Side Form Validation	100% Implemented	Regex verification & toast system